Secure File Storage

Segurança de Software

Rúben Silva pg57900

21 May 2026

Problem & Motivation

The Challenge:

- Os utilizadores armazenam ficheiros na cloud sem confiar no servidor
- O comprometimento do backend NÃO deve expor ficheiros
- Multi-Tenant deve impedir vazamentos entre utilizadores

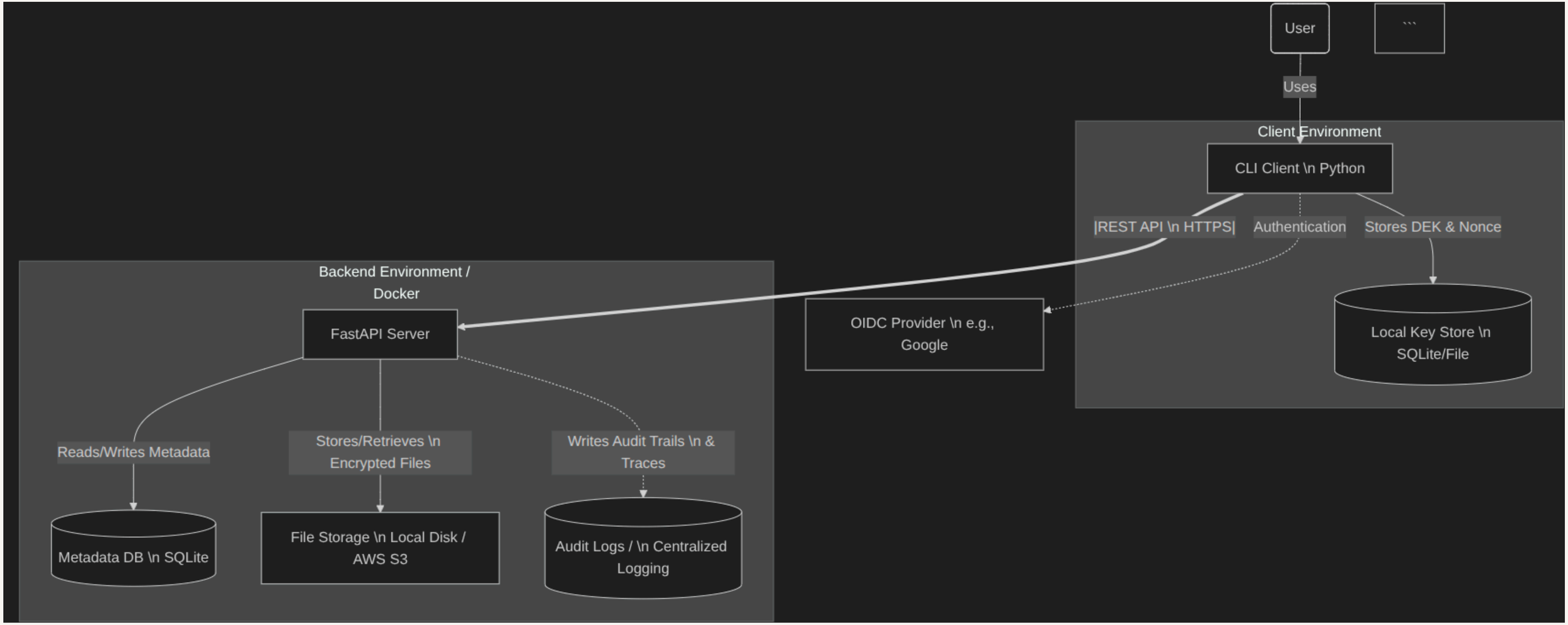
Approach:

1. Full Privacy Mode
    a. Client-side encryptation
2. Secure Development lifecycle
3. Zero Knowledge:
    a. O Backend só vê dados encriptados

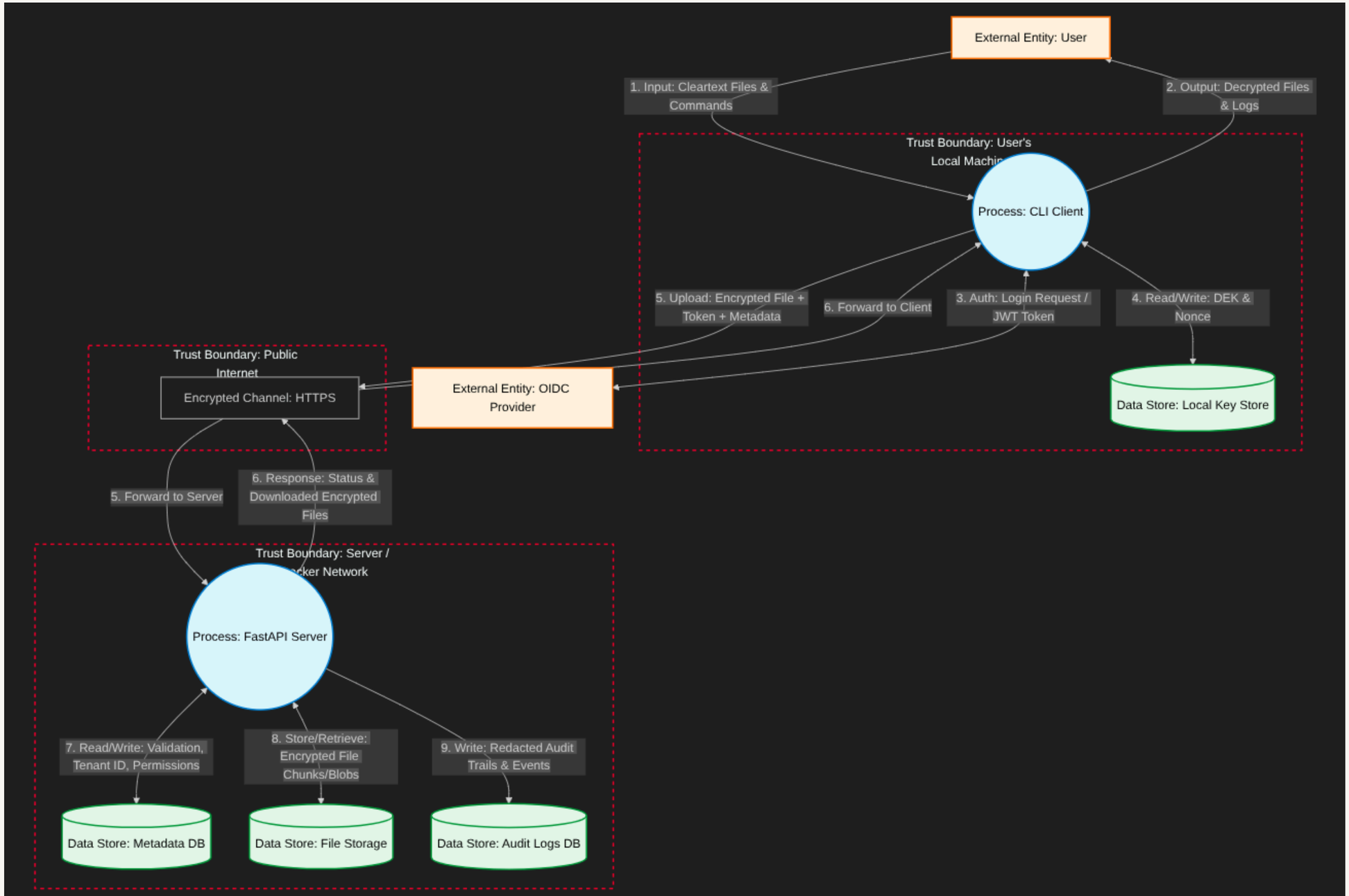
Req. do Enunciado

1. **Objetivo:** armazenamento privado E2E
2. **Backend:** FastAPI + SQLAlchemy
3. **Cliente:** CLI (AES-GCM local)
4. **Auth:** OIDC (Google JWT)
5. **Multitenancy:** isolamento por tenant
6. **Partilha:** shares + links anónimos (TTL)
7. **Chaves:** DEK por ficheiro
8. **Auditoria:** logs com request-ID + redacção
9. **Proteções:** rate-limit, limites de upload, hardening ....
10. **Deploy:** container não-root, segredos via env

Arquitetura do Sistema



Data Flow Diagram - Threat Model



Threat Model - Stride

Threat Category	Potential Threat in Our System	Affected Component / Flow	Mitigation Strategy
Spoofing	An attacker intercepts or forges an OIDC token to impersonate a legitimate user and access their files.	Flow 3 & 5 (Public Internet)	Strict cryptographic validation of the OIDC JWT (signature, audience, and expiration) on the FastAPI server.
Tampering	A malicious actor or compromised network alters the encrypted file chunks in transit, or modifies the metadata directly in the SQLite database.	Flow 5 (Internet) & Flow 7 (Backend DB)	Use of TLS (HTTPS) for transit. Client-side encryption using **AES-GCM** ensures data integrity (tamper-evident). Use of SQLAlchemy ORM to prevent SQL Injection.
Repudiation	A user maliciously deletes a shared file and denies doing so, or support staff accesses files without traceability.	P2 (FastAPI Server)	Comprehensive centralized audit logging. All critical actions (upload, delete, permission changes) must be logged with a unique `Request-ID`, Timestamp, and the user's OIDC Subject ID.
Information Disclosure	The server or a cloud provider (AWS S3) inspects the file contents. Another tenant accesses metadata belonging to a different user.	Flow 8 (File Storage) & DS2 (Metadata DB)	**Full Privacy Mode:** Client-side encryption ensures the server only sees ciphertext. Strict **Multi-Tenancy isolation** logic ensures DB queries are always scoped to the authenticated user's Tenant ID. Filenames are anonymized (UUIDs).
Denial of Service (DoS)	An attacker spams the API with massive file uploads or thousands of requests, crashing the server or exhausting storage.	P2 (FastAPI Server) & DS3 (File Storage)	Implement API Rate Limiting and strict payload size limits for file uploads.
Elevation of Privilege	A standard user attempts to bypass permissions to access another tenant's files, or attempts to access the Admin Portal endpoints.	P2 (FastAPI Server)	Strict authorization checks on every endpoint. Role-Based Access Control (RBAC) implementation, ensuring admin features require specific OIDC roles.

Threat Model - Security Requirements

1. **Strict Tenant Isolation:** Every database query interacting with file metadata MUST include a filter for the authenticated user's Tenant ID. Cross-tenant data access is strictly forbidden unless explicitly granted via the sharing system.
2. **Zero-Knowledge Storage (Full Privacy Mode):** The server MUST NOT receive or store plaintext files. All files MUST be encrypted client-side using AES-GCM before transmission.
3. **Metadata Anonymization:** The server MUST NOT know the original filenames or extensions. Files must be stored on disk/S3 using randomly generated UUIDs.
4. **Strong Authentication & Authorization:** All API endpoints (except public share links) MUST require a valid OIDC JWT. Permissions (Read/Write) must be validated on every request.
5. **Secure Cryptographic Practices:** The client MUST ensure unique Nonce generation for every AES-GCM encryption operation to prevent catastrophic key reuse vulnerabilities.
6. **Input Validation & Safe File Handling:** All inputs MUST be sanitized. The server must be protected against Path Traversal and OS Command Injection by abstracting the file system layer and never using user-provided input for file paths.
7. **Secure Audit Logging:** The system MUST implement centralized logging. Logs MUST enforce redaction rules to ensure no secrets, PII, or tokens are ever leaked.
8. **Secure Deployment:** The Docker deployment MUST NOT contain hardcoded credentials. All secrets must be injected via environment variables (`ENV`), and containers MUST run as non-root users.

Endpoints & Capabilities

- POST /files/ → **Upload Encrypted**
- GET /files/ → **List Owned** + *Shared Files*
- GET /files/{file_id}/download → **Download Specific File**
- PUT /files/{file_id} → **Update File** *(New Version)*
- DELETE /files/{file_id} → **Delete file** + *metadata cleanup*

- GET /files/{file_id}/permissions → **View Permissions**
- PUT /files/{file_id}/permissions → **View Permissions**

- POST /files/{file_id}/share → **Share** with other user
- POST /files/{file_id}/anonymous-link → **Create Anonymous Link** (TTL-Protected)
- GET files/anon/{link_id} → **Donwload via Anonymous Link**

- GET /health → **Saber se o server está vivo**
- POST auth/verify → **Verificar o OIDC JWT Token**

- python cli/main.py import-key <file_id> <name> → **Importa uma DEK e guarda-a no cofre local encriptado**
- python cli/main.py migrate-vault → **Converte um plaintext para formato encriptado com passphrase**
- python cli/main.py erase-key <file_id> → **Apaga apenas a DEK local desse ficheiro**

Defensive Secuirty Engineering

Autenticação

Auth.py

Nome da Feature	O que ela Faz
Autenticação Bearer obrigatória	Obriga cada pedido autenticado a trazer um token Bearer no header Authorization antes de entrar na API.
Validação OIDC com Google JWKS	Confirma a identidade do utilizador validando assinatura, audience e claims do JWT com chaves públicas da Google.
Seleção de chave por kid com cache JWKS	Procura a chave pública correta pelo `kid`, mantém uma cache local e faz refresh controlado quando a chave ainda não está disponível.
Fail-closed se GOOGLE_CLIENT_ID faltar	Bloqueia a API quando a configuração obrigatória de produção não existe, em vez de aceitar tokens sem validação forte.

Defensive Secuirty Engineering

Logger

Logger.py

Nome da Feature	O que ela Faz
Thread-safe request ID context	Guarda um request id por contexto assíncrono para não misturar pedidos concorrentes.
Redaction de segredos nos logs	Remove Bearer tokens e emails dos logs antes de estes serem escritos no terminal ou no ficheiro de auditoria.
Logs com request ID	Escreve cada evento com ReqID para permitir rastreabilidade e auditoria de ponta a ponta.
Rotating log file handler	Limita o tamanho do ficheiro de logs e mantém backups para evitar exaustão de disco por crescimento ilimitado.

Defensive Secuirty Engineering

Modelos

Models.py

Nome da Feature	O que ela Faz
Cascade delete de shares/links	Remove automaticamente relações dependentes quando um ficheiro é apagado.
File ID por UUID	Garante que o identificador do ficheiro é aleatório e não revela o nome original.
Ficheiros anonimizados	O storage trabalha com UUIDs e não preserva o nome original do utilizador.
Permissão do ficheiro por defeito	Cria ficheiros novos já com permissões seguras por omissão, evitando estados indefinidos.
Token anónimo criptograficamente forte	Gera IDs de links anónimos com `secrets.token_urlsafe(32)`, dando entropia forte ao token.
Link anónimo read-only por defeito	Faz com que os links anónimos comecem em leitura, reduzindo o risco de abuso acidental.
Expiração automática de links anónimos	Faz os links anónimos expirar por omissão após 7 dias, limitando a superfície de ataque pública.

Defensive Secuirty Engineering server 1

Main.py

Nome da Feature	O que ela Faz
Tenant isolation por owner/shared	Só devolve ficheiros do dono ou partilhas explícitas; não há fallback cross-tenant.
Listagem explicitamente separada	Separa owned_files de shared_with_me para tornar a autorização transparente e auditável.
Download protegido por ownership/share	Só permite download quando o utilizador é dono ou foi partilhado explicitamente.
Owner-only delete	Só o dono pode apagar um ficheiro; outros pedidos recebem 404 para não revelar demasiado.
Prevenção de auto-partilha	Bloqueia a partilha de um ficheiro com o próprio utilizador para evitar confusão e erros de autorização.
Permissões validadas	Só aceita `read` ou `read/write` nas atualizações de permissão, rejeitando valores inválidos.
TTL bounded validation	Limita a retenção a valores plausíveis e rejeita TTL inválidos ou absurdos.
Normalização UTC	Evita bugs de comparação entre datas naive e aware ao tratar expirações.
Remoção de metadata órfã	Elimina registos stale e devolve 410 quando o ficheiro já não existe no storage.
Rollback de metadata em falha de storage	Reverte o registo na base de dados se a escrita do ficheiro falhar.

Defensive Secuirty Engineering server 2

Upload em stream limitado	Processa uploads em chunks para evitar consumo descontrolado de RAM.
Limite máximo de upload	Rejeita uploads grandes com 413 para impedir exaustão de armazenamento.
Rate limiting por janela deslizante	Reduz abuso e DoS por spam de pedidos num intervalo curto.
IP-aware behind proxy	Usa headers de proxy para tentar identificar o IP real do cliente.
Anonymous link creation segura	Gera links públicos de acesso sem expor a chave do ficheiro.
Anonymous download with expiry check	Bloqueia links expirados e limpa metadados órfãos quando o ficheiro já não existe.
Request ID refletido na resposta	Permite correlacionar pedidos cliente-servidor em debug e auditoria.
Self-sharing rejection	Impede que o utilizador partilhe um ficheiro consigo mesmo.
Validation of sharing targets	Garante que a relação de partilha é criada só quando necessário e de forma explícita.
Health check público sem autenticação	Permite monitorização de liveness/readiness sem bloquear por autenticação e sem expor dados sensíveis.

Defensive Secuirty Engineering

Encriptação

Crypto.py

Nome da Feature	O que ela Faz
Client-side AES-256-GCM	Cifra os ficheiros no cliente antes de chegarem ao servidor.
Nonce novo em cada cifragem	Evita reuse catastrófico de nonce no AES-GCM.
Padding em blocos fixos	Reduz leakage do tamanho real do ficheiro para o servidor e para observadores.
Envelope com tamanho original	Permite remover o padding e recuperar o plaintext correto depois da desencriptação.

Defensive Secuirty Engineering

CLI

CLI Main.py

Nome da Feature	O que ela Faz
Uma DEK por ficheiro	Reduz o impacto se uma chave individual for comprometida.
Cofre local encriptado em repouso	Guarda DEKs e metadados num ficheiro local cifrado.
Chave do cofre derivada com Scrypt	Endurece o cofre contra brute force e roubo do ficheiro local.
Salt + nonce + ciphertext no cofre	Reencripta o cofre em cada gravação com material fresco.
Criptographic erasure por file_id	Apaga a chave local associada a um ficheiro específico.
Chave nunca sai do cliente	O servidor recebe apenas ciphertext e nunca a DEK.
Troca de chave fora da API	Força a partilha da DEK por canal seguro, não pela API.
Erasure best-effort no delete	Remove a chave local depois do delete no servidor.

Defensive Secuirty Engineering

Deploy

Deploy

Nome da Feature	O que ela Faz
Non-root container execution	O container corre com um utilizador não privilegiado.
Secrets via ENV only	As credenciais e parâmetros sensíveis são injetados por ambiente, não hardcoded.
Mandatory GOOGLE_CLIENT_ID in compose	O deployment falha se a audience OIDC não for fornecida.
tmpfs for /tmp	Evita persistência de dados temporários em disco.
no-new-privileges container flag	Bloqueia escalada de privilégios dentro do container.
cap_drop ALL	Remove capacidades Linux do container para reduzir a superfície de ataque.
Database directory owned by appuser	Prepara o filesystem do container para execução sem root.

Defensive Secuirty Engineering

Tests

Tests

Nome da Feature	O que ela Faz
Security tests for tenant isolation	Valida que o cross-tenant access é bloqueado.
Security tests for permission bypass	Valida que shared users sem write não conseguem alterar.
Security tests for misuse	Valida rejeição de inputs e fluxos inválidos.

Escolha Criptográfica

AES-GCM:

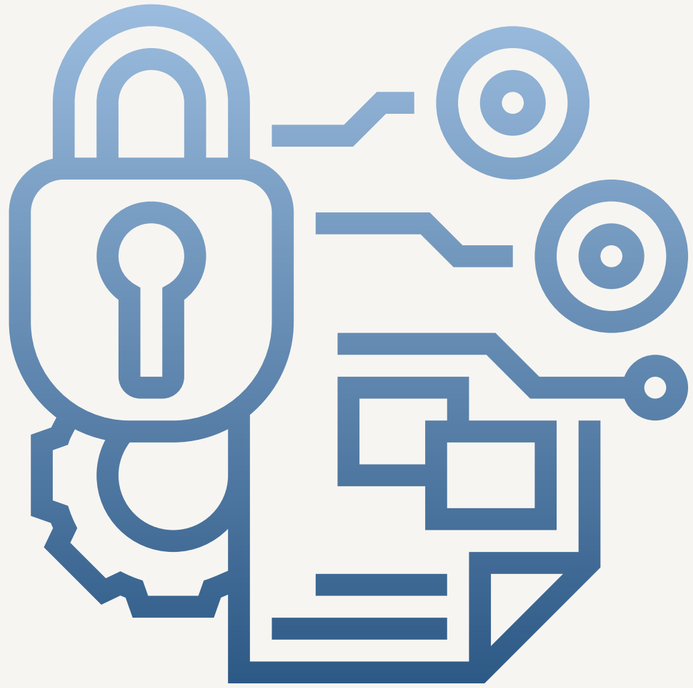
- Confidentiality & Integrity
- key 256-bit
- Nonce 96-bit

Per-File DEKs:

- Cada Ficheiro tem a sua chave pra evitar leaks que comprometam a BD toda de uma vez

Encriptação com padding:

- Esconder o tamanho real do ficheiro antes da encriptação



Automação de Testes de Sec.

Test	Validates
Test 1-2	Tenant isolation: User A cannot see User B's files
Test 3-4	Permission bypass: read-only users cannot update files
Test 5-6	Anonymous link expiration (TTL enforcement)
Test 7-8	File versioning: updates create new versions
Test 9	Oversized upload (>50 MB) rejected with 413
Test 10	Rate limiting blocks 61st request (60 req/min)
Test 11	Malformed JWT rejected (403 Forbidden)
Test 12	File deletion removes metadata + storage
Test 13	Audit log captures all critical actions

CI/CD

Pipeline:

1. **pytest** - 13 Security Tests
2. **bandit** - Static Analysis
3. **pip-audit** - Dependency Vulnerability Scan
4. **detect-secrets** - Credential Leaking

Deploy & Hardening

Deploy (secure by default)

```
FROM python:3.12-slim
RUN useradd -m -u 1000 secureuser  # Non-root user
USER secureuser
ENV GOOGLE_CLIENT_ID=${GOOGLE_CLIENT_ID}  # Secrets via ENV
```

Security Features:

- Non-root user
- env contém os secrets
- API corre na porta 8000 (non-privileged)

Secure File Storage

Segurança de Software

Rúben Silva pg57900

21 May 2026